

1. DOM (Document Object Model) Introduction

DOM (Document Object Model) ek programming interface hai jo HTML document ko ek tree structure ke form me represent karta hai.

Iska use JavaScript ke through HTML elements ko access, modify aur manipulate karne ke liye kiya jata hai.

Key Points :

- DOM browser dwara banaya gaya ek object model hai.
- Ye HTML document ka ek in-memory representation hai.
- Isse JavaScript ke through HTML aur CSS ko dynamic banaya jata hai.
- DOM ka har element ek object hota hai.
- DOM ke through content, structure aur style sabhi ko change kiya ja sakta hai.

Why DOM is Important ?

1. Web page ko dynamic aur interactive banata hai.
2. User actions (events) ko handle karne me madad karta hai.
3. HTML elements ko create, delete, update karne ki suvidha deta hai.
4. Real time applications banane ke liye essential hai.

Example :

HTML Code :

```
<!DOCTYPE html>
<html>
  <head>
    <title>BK LearnX</title>
  </head>
  <body>
    <h1 id="title">Welcome to BK LearnX</h1>
    <p class="para">Learn | Practice | Achieve</p>
    <button id="btn">Click Me</button>
  </body>
</html>
```

JavaScript (DOM use) :

```
// Get element by id
let heading = document.getElementById("title");
heading.innerHTML = "Welcome to BK LearnX!";

// Get element by class
let para = document.getElementsByClassName("para")[0];
para.style.color = "blue";

// Get element by id and change content
document.getElementById("btn").innerText = "Clicked!";
```

Note :

- * DOM ki help se hum HTML page ke content, structure, style aur behavior sabko control kar sakte hain.

2. DOM Tree / Structure

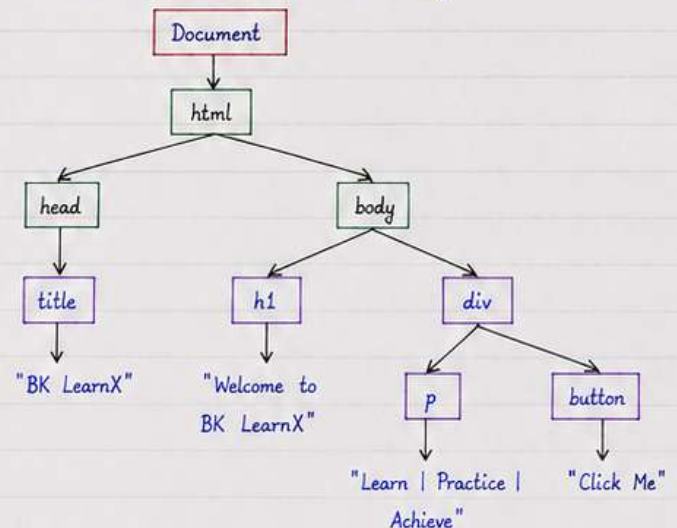
Jab browser HTML document ko load karta hai to wo us document ka ek tree structure banata hai jise DOM Tree kehte hain.

Is tree me har HTML tag ek node (object) hota hai.

Example HTML Code :

```
<!DOCTYPE html>
<html>
  <head>
    <title>BK LearnX</title>
  </head>
  <body>
    <h1 id="title">Welcome to BK LearnX</h1>
    <div class="content">
      <p class="para">Learn | Practice | Achieve</p>
      <button id="btn">Click Me</button>
    </div>
  </body>
</html>
```

Is HTML ka DOM Tree kuch is prakar hoga :



Key Points :

- DOM Tree ki root node hoti hai → Document.
- Document ke under ek hi html element hota hai.
- html ke under head aur body hote hain.
- Body ke andar humare visible elements (tags) hote hain.
- Har tag ek node hai aur unke beech parent-child relationship hoti hai.

Types of Nodes in DOM :

1. Document Node → Pure document ko represent karta hai.
2. Element Node → HTML tags (jaise h1, div, p, button).
3. Text Node → Text content (jaise "Welcome...").
4. Attribute Node → Attributes (jaise id, class).
5. Comment Node → HTML comments.



3. Selecting Elements in DOM

DOM me kisi element ko select karne ke liye JavaScript me kai methods diye gaye hain. In methods ki help se hum page ke elements ko access kar sakte hain.

(i) getElementById()

- Ye method id attribute ke basis par element select karta hai.
- Isme hamesha single element return hota hai.

Syntax :

```
document.getElementById("id");
```

Example :

HTML : `<h1 id="title">Hello</h1>`

JS :

```
let el = document.getElementById("title");  
console.log(el); // <h1 id="title">Hello</h1>
```

(ii) getElementsByClassName()

- Ye method class name ke basis par elements select karta hai.
- Isme HTMLCollection return hoti hai (sabhi matched elements).

Syntax :

```
document.getElementsByClassName("class");
```

Example :

HTML : `<p class="para">One</p>
<p class="para">Two</p>`

JS :

```
let list = document.getElementsByClassName("para");  
console.log(list); // HTMLCollection(2) [p, p]
```

(iii) getElementsByTagName()

- Ye method tag name ke basis par elements select karta hai.
- Isme bhi HTMLCollection return hoti hai.

Syntax :

```
document.getElementsByTagName("tag");
```

Example :

HTML : `<p>First</p>
<p>Second</p>
<p>Third</p>`

JS :

```
let paras = document.getElementsByTagName("p");  
console.log(paras); // HTMLCollection(3) [p, p, p]
```

Key Points :

- `getElementById()` → id se element select karta hai (Single).
- `getElementsByClassName()` → class se sabhi elements select karta hai (HTMLCollection).
- `getElementsByTagName()` → tag se sabhi elements select karta hai (HTMLCollection).

4. querySelector() & querySelectorAll()

Ye modern methods CSS selector ka use karke elements select karne ke liye use hote hain.

(i) querySelector()

- Ye method CSS selector ke basis par pehla matched element select karta hai.
- Ye single element return karta hai.

Syntax :

```
document.querySelector("css selector");
```

Example :

HTML :

```
<h1 id="title">Welcome</h1>  
<p class="para">Hello</p>  
<p class="para">Hi</p>
```

JS :

```
let el = document.querySelector(".para"); // class se  
console.log(el.innerText); // Hello (pehla matched)
```

```
let el2 = document.querySelector("#title"); // id se  
console.log(el2.innerText); // Welcome
```

(ii) querySelectorAll()

- Ye method CSS selector ke basis par sabhi matched elements select karta hai.
- Ye NodeList return karta hai (sabhi matched elements).

Syntax :

```
document.querySelectorAll("css selector");
```

Example :

HTML :

```
<h1 id="title">Welcome</h1>  
<p class="para">Hello</p>  
<p class="para">Hi</p>
```

JS :

```
let list = document.querySelectorAll(".para");  
console.log(list); // NodeList(2) [p.para, p.para]
```

```
list.forEach((el) => {  
    console.log(el.innerText);  
});
```

// Output :

// Hello

// Hi

Key Points :

- `querySelector()` → pehla matched element return karta hai.
- `querySelectorAll()` → sabhi matched elements return karta hai.
- CSS selectors ka use karke id (#), class (.), tag, attribute sab select kiye ja sakte hain.
- Modern websites me in dono methods ka use zyada hota hai.



BK LearnX
Learn | Practice | Achieve

5. innerHTML, textContent & innerText

In properties ka use karke hum HTML content ya text ko read aur change kar sakte hain.

(i) innerHTML

- Ye element ke andar ke HTML content ko get ya set karta hai (tags ke saath).
- Isme agar hum koi HTML tag likhte hain to wo render bhi ho jata hai.

Syntax :

```
element.innerHTML
element.innerHTML = "<b>Hello</b>";
```

Example :

```
HTML : <div id="box">Hello <b>World</b></div>
JS :
let box = document.getElementById("box");
console.log(box.innerHTML); // Hello <b>World</b>
box.innerHTML = "<i>Welcome to BK LearnX</i>";
// Output: <i>Welcome to BK LearnX</i> (italic me show hoga)
```

(ii) textContent

- Ye element ke andar ke sirf text content ko get ya set karta hai.
- Agar HTML tag hai to wo ojo dikhata nahi honge, sirf text dikhega.

Syntax :

```
element.textContent
element.textContent = "Hello";
```

Example :

```
HTML : <div id="box">Hello <b>World</b></div>
JS :
let box = document.getElementById("box");
console.log(box.textContent); // Hello World
box.textContent = "Welcome to BK LearnX";
// Output: Sirf text set hoga, koi tag interpret nahi hoga.
```

(iii) innerText

- Ye visible text ko get ya set karta hai.
- Ye CSS styles (jaise display:none) ko bhi consider karta hai.

Syntax :

```
element.innerText
element.innerText = "Hello";
```

Example :

```
HTML : <div id="box">Hello <b>World</b></div>
JS :
let box = document.getElementById("box");
console.log(box.innerText); // Hello World (visible text)
box.innerText = "Welcome to BK LearnX";
// Output: Sirf visible text set hoga.
```

Key Points :

- innerHTML → HTML tags ke saath content.
- textContent → Sirf text, tags as text.
- innerText → Sirf visible text (CSS consider karta hai).

6. Changing CSS / Styles

DOM ke through hum kisi element ki CSS properties ko get ya set (change) kar sakte hain.

(i) style Property

- Har HTML element ka ek style object hota hai.
- Is style object ki help se hum inline CSS set karte hain.

Syntax :

```
element.style.property = "value";
```

Example : Change text color and background

```
HTML : <p id="para">Hello BK LearnX</p>
JS :
let p = document.getElementById("para");
p.style.color = "blue"; // text color blue
p.style.backgroundColor = "yellow"; // background yellow
p.style.fontSize = "20px"; // font size
p.style.padding = "10px"; // inner space
```

(ii) Get Computed Styles

- Agar hum kisi element ki current/actual CSS value jaanna chahte hain (chahe wo external/internal/inline kahin se bhi aayi ho), to getComputedStyle() use karte hain.

Syntax :

```
window.getComputedStyle(element).property
```

Example :

```
HTML : <h2 id="title" style="color: green;">BK LearnX</h2>
JS :
let h2 = document.getElementById("title");
let color = window.getComputedStyle(h2).color;
console.log(color); // rgb(0, 128, 0) (browser format)
```

(iii) Common CSS Properties (JS style name)

CSS Property	JS Style Name
background-color	backgroundColor
font-size	fontSize
text-align	textAlign
margin-top	marginTop
padding-left	paddingLeft
border-radius	borderRadius
display	display
visibility	visibility
width	width
height	height

Key Points :

- element.style.property → inline CSS set karta hai.
- getComputedStyle(element).property → actual computed value return karta hai.
- JS style names me hyphen(-) ki jagah camelCase use hota hai.



BK LearnX
Learn | Practice | Achieve

9. addEventListener() & Event Object

`addEventListener()` method ka use karke hum kisi element par event ko listen kar sakte hain aur event use hone par ek function execute hota hai. Event object hume event se related information deta hai.

(i) addEventListener()

- Ye method element par event listener attach karta hai.
- Isme hum multiple events add kar sakte hain.
- Inline event se behtar aur recommended method hai.

Syntax :

```
element.addEventListener("event", function);
```

Example :

```
HTML : <button id="btn">Click Me</button>
       <p id="msg"></p>
```

```
JS : document.getElementById("btn").addEventListener("click",
function() {
    document.getElementById("msg").innerText = "Button Clicked!";
});
```

(ii) Multiple Events Example

```
let btn = document.getElementById("btn");
btn.addEventListener("mouseover", function(){
    btn.style.backgroundColor = "lightblue";
});
btn.addEventListener("mouseout", function(){
    btn.style.backgroundColor = "";
});
```

(iii) Event Object

- Event object ek built-in object hai jo event se related details provide karta hai.
- Is object ko function ke parameter me lete hain (usually e or event).
- Isse hum pata kar sakte hain ki kaunsa key press hua, mouse ki position kya hai, kis element par event trigger hua, etc.

Syntax :

```
element.addEventListener("event", function(event) {
    // event object use here
});
```

Example :

```
let btn = document.getElementById("btn");
btn.addEventListener("click", function(event) {
    console.log(event);           // pura event object
    console.log(event.type);      // event type (click)
    console.log(event.target);    // jis element par click hua
});
```

Key Points :

- `addEventListener()` better approach hai (clean & flexible).
- Event object se event ki details milti hain.
- Same element par multiple events attach kiye ja sakte hain.
- `event.target` se pata chalta hai ki event kis element par trigger hua.

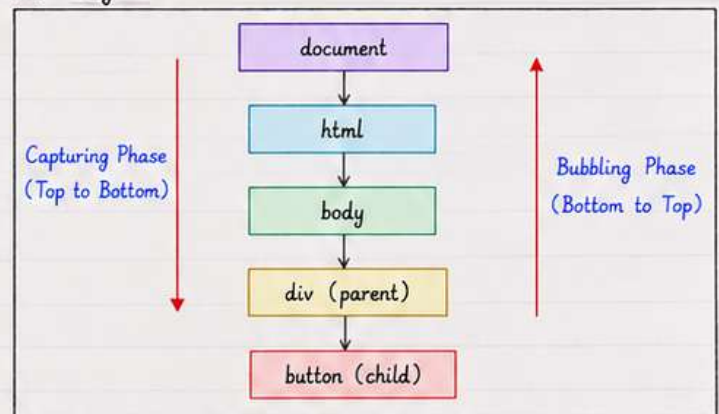
10. Event Bubbling and Capturing

Jab kisi element par event trigger hota hai to wo sirf usi element par nahi rukta, balki uske parent elements tak bhi propagate karta hai. Is propagation ko Event Flow kehte hain. Iske do phases hote hain - Capturing Phase aur Bubbling Phase.

(i) Event Propagation (Flow)

1. **Capturing Phase** : Event root se target element tak aata hai (upar se niche).
2. **Target Phase** : Event target element par trigger hota hai.
3. **Bubbling Phase** : Event target se wapas root tak jata hai (niche se upar).

(ii) Diagram :



(iii) Example : Event Bubbling

HTML :

```
<div id="parent">
  <button id="child">Click</button>
</div>
```

JS :

```
document.getElementById("parent").addEventListener("click", function() {
    console.log("Parent Clicked (Bubbling)");
});
document.getElementById("child").addEventListener("click", function() {
    console.log("Child Clicked");
});
```

(iv) Example : Capturing (useCapture = true)

```
document.getElementById("parent").addEventListener("click", function() {
    console.log("Parent Clicked (Capturing)");
}, true); // true = capturing phase

document.getElementById("child").addEventListener("click", function() {
    console.log("Child Clicked");
}, true);
```

Key Points :

- Event propagation ke do phases hote hain - Capturing (top → bottom) aur Bubbling (bottom → top).
- By default, events bubbling phase me kaam karte hain.
- `addEventListener()` me third parameter true dene se capturing phase use hota hai.
- `event.stopPropagation()` se hum event ko aage propagate hone se rok sakte hain.



BK LearnX
Learn | Practice | Achieve

11. classList in DOM

classList ek modern property hai jiska use hum kisi element ke CSS classes ko add, remove, toggle ya check karne ke liye karte hain. Ye bahut easy aur simple way provide karta hai classes ko handle karne ka.

classList Methods :

Method	Description
add()	Nayi class add karta hai.
remove()	Existing class remove karta hai.
toggle()	Agar class nahi hai to add karta hai, agar hai to remove karta hai.
contains()	Check karta hai ki class exist karti hai ya nahi (true/false return karta hai).
replace()	Ek class ko doosri class se replace karta hai.

(i) add() Method

- Is method se hum kisi element me nayi class add karte hain.
- Agar class pehle se exist karti hai to duplicate add nahi hoti.

Syntax :

```
element.classList.add("className");
```

Example :

```
document.getElementById("box").classList.add("active");
```

(ii) remove() Method

- Is method se hum kisi element se class remove karte hain.
- Agar class exist nahi karti hai to koi error nahi aata.

Syntax :

```
element.classList.remove("className");
```

Example :

```
document.getElementById("box").classList.remove("active");
```

(iii) toggle() Method

- Agar class nahi hai to add kar deta hai.
- Agar class hai to remove kar deta hai.

Syntax :

```
element.classList.toggle("className");
```

Example :

```
document.getElementById("box").classList.toggle("dark");
```

(iv) contains() Method

- Ye method check karta hai ki element me class hai ya nahi.
- Return value → true (hai) ya false (nahi hai).

Syntax :

```
element.classList.contains("className");
```

Example :

```
let hasClass = document.getElementById("box").  
classList.contains("active");  
console.log(hasClass); // true or false
```

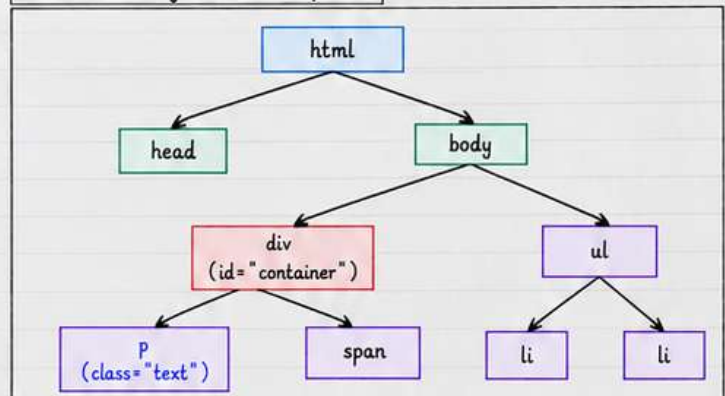
Key Points :

- classList modern aur easy way hai classes ko handle karne ka.
- Hum ek element me multiple classes add/remove kar sakte hain.
- toggle() dark/light mode jaise features me bahut useful hai.
- contains() condition check karne ke liye use hota hai.

12. DOM Traversing

DOM Traversing ka matlab hai document ke elements ke beech move karna. Iski help se hum kisi element ke parent, child, sibling ya ancestors/descendants tak access kar sakte hain.

DOM Traversing Tree (Example)



Important Traversing Properties :

Property	Description
parentElement	Parent element ko return karta hai.
children	Direct child elements (HTMLCollection).
firstElementChild	Pehla child element.
lastElementChild	Last child element.
nextElementSibling	Agla sibling element.
previousElementSibling	Pehla (previous) sibling element.
closest()	Sabse paas ka ancestor element (condition ke saath).

(i) parentElement

Example :

```
let child = document.getElementById("text");  
let parent = child.parentElement;  
console.log(parent); // <div id="container">
```

(ii) children, firstElementChild, lastElementChild

Example :

```
let parent = document.getElementById("container");  
console.log(parent.children); // all child elements  
console.log(parent.firstElementChild); // first child  
console.log(parent.lastElementChild); // last child
```

(iii) nextElementSibling & previousElementSibling

Example :

```
let item = document.querySelector(".text");  
console.log(item.nextElementSibling); // next sibling  
console.log(item.previousElementSibling); // previous sibling
```

(iv) closest()

Example :

```
let el = document.querySelector(".text");  
let parentDiv = el.closest("div");  
console.log(parentDiv); // nearest div ancestor
```

Key Points :

- DOM Traversing se hum page ke elements ke beech easily navigate kar sakte hain.
- Ye dynamic websites banane me bahut useful hota hai.
- closest() method tab use karte hain jab hume condition ke saath nearest parent chahiye.



BK LearnX
Learn | Practice | Achieve

13. Form & Input Handling in DOM

Form HTML ka ek important part hai jis se hum user se input le sakte hain. DOM ka use karke hum form ke data ko access, validate aur handle kar sakte hain.

(i) Accessing Form Elements

- Hum document methods se form ke input fields ko select karte hain.
- value property se input ki value milti hai.

Syntax :

```
let input = document.getElementById("inputId");
let value = input.value;
```

Example :

HTML : `<input type="text" id="name" placeholder="Enter Name">`

JS :

```
let name = document.getElementById("name").value;
console.log(name); // user input value
```

(ii) Handling form submission

- submit event ka use karke form submit hone par action perform kar sakte hain.
- event.preventDefault() se page refresh hone se rok sakte hain.

Example :

HTML :

```
<form id="myForm">
  <input type="text" id="name" placeholder="Enter Name">
  <button type="submit">Submit</button>
</form>
```

JS :

```
document.getElementById("myForm").addEventListener("submit",
  function(event){
    event.preventDefault(); // page refresh nahi hoga
    let name = document.getElementById("name").value;
    console.log("Name : " + name);
  });
```

(iii) Form Validation (Basic)

- Hum JavaScript se form validation kar sakte hain.
- Agar input empty hai to error message dikha sakte hain.

Example :

```
let name = document.getElementById("name").value;
if(name == ""){
  alert("Please enter your name");
} else {
  alert("Form submitted successfully!");
}
```

(iv) Common Input Types

Input Type	Use
text	Single line text input
password	Hidden text (password)
email	Email input
number	Numeric input
checkbox	Multiple choice (true/false)
radio	Single choice from options
date	Date selection
file	File upload
submit	Form submit button
reset	Form reset button

Key Points :

- value property se input ki value milti hai.
- submit event form handle karne ke liye use hota hai.
- event.preventDefault() se page refresh rukta hai.
- Validation JavaScript se easily ki ja sakti hai.

14. DOM Mini Projects

DOM ke concepts ko achhe se samajhne ke liye chote-chote projects banana bahut useful hota hai.

(i) Project 1 : To-Do List

User tasks add kare, delete kare aur list ko dynamically update kare.

HTML + JS (main part) :

```
let input = document.getElementById("task");
let btn = document.getElementById("addBtn");
let list = document.getElementById("taskList");

btn.addEventListener("click", function(){
  if(input.value == "") return;
  let li = document.createElement("li");
  li.innerText = input.value;
  let del = document.createElement("button");
  del.innerText = "Delete";
  del.onclick = function(){ li.remove(); };
  li.appendChild(del);
  list.appendChild(li);
  input.value = "";
});
```

Output (UI) :

My To-Do List

Add

- Read Book Delete
- Learn JavaScript Delete
- Build Project Delete

(ii) Project 2 : Image Slider

Images ko next/previous button se change kare.

```
let images = [
  "img1.jpg", "img2.jpg", "img3.jpg"
];
let index = 0;
let img = document.getElementById("sliderImg");

function showImage(){
  img.src = images[index];
}

function next(){
  index = (index + 1) % images.length;
  showImage();
}

function prev(){
  index = (index - 1 + images.length) % images.length;
  showImage();
}
```

Output (UI) :



(iii) Project 3 : Form Validation

Simple form with name, email and validation.

```
document.getElementById("myForm").addEventListener(
  "submit", function(e){
    e.preventDefault();
    let name = document.getElementById("name").value;
    let email = document.getElementById("email").value;
    if(name == "" || email == ""){
      document.getElementById("msg").innerText =
        "Please fill all fields!";
    } else {
      document.getElementById("msg").innerText =
        "Form submitted successfully!";
    }
  });
```

Output (UI) :

User Registration

Submit

Form submitted successfully!

(iv) Project 4 : Theme Toggle (Bonus)

One button se light/dark mode change kare.

```
let btn = document.getElementById("toggleBtn");
btn.addEventListener("click", function(){
  document.body.classList.toggle("dark");
  if(document.body.classList.contains("dark")){
    btn.innerText = "Light Mode";
  } else {
    btn.innerText = "Dark Mode";
  }
});
```

Output (UI) :



Key Points :

- Projects se DOM concepts ka practical use samajh aata hai.
- Chote projects se problem solving skill improve hoti hai.
- In projects me element selection, event handling, DOM manipulation sab use hue.
- Aap isko aur improve karke apne portfolio me add kar sakte hain.



BK LearnX
Learn | Practice | Achieve